Introduction to Lattice-based Cryptography

October 29, 2026

Blockstream Research

research.blockstream.com

github.com/BlockstreamResearch/zk-book



Plan

1 Unstructured Lattices

2 Short Integer Solution (SIS)

3 Learning With Errors (LWE)

4 Structured Lattices

What will we learn today?

- What a lattice is and how to measure it: fundamental domain, covolume, successive minima.

What will we learn today?

- What a lattice is and how to measure it: fundamental domain, covolume, successive minima.
- Which lattice problems are believed to be hard, even for **quantum** adversaries: SVP_γ, CVP_γ, BDD_γ, GapSVP_γ.

What will we learn today?

- What a lattice is and how to measure it: fundamental domain, covolume, successive minima.
- Which lattice problems are believed to be hard, even for **quantum** adversaries: SVP_γ, CVP_γ, BDD_γ, GapSVP_γ.
- The two workhorses of lattice-based cryptography: **Short Integer Solution (SIS)** and **Learning With Errors (LWE)**.

What will we learn today?

- What a lattice is and how to measure it: fundamental domain, covolume, successive minima.
- Which lattice problems are believed to be hard, even for **quantum** adversaries: SVP_γ, CVP_γ, BDD_γ, GapSVP_γ.
- The two workhorses of lattice-based cryptography: **Short Integer Solution (SIS)** and **Learning With Errors (LWE)**.
- What we can build from them: hash functions, commitments, public-key encryption.

What will we learn today?

- What a lattice is and how to measure it: fundamental domain, covolume, successive minima.
- Which lattice problems are believed to be hard, even for **quantum** adversaries: SVP_γ, CVP_γ, BDD_γ, GapSVP_γ.
- The two workhorses of lattice-based cryptography: **Short Integer Solution (SIS)** and **Learning With Errors (LWE)**.
- What we can build from them: hash functions, commitments, public-key encryption.
- Why practical schemes (ML-KEM, ML-DSA) work over the polynomial ring $\mathcal{R}_q = \mathbb{Z}_q[T]/(T^n + 1)$.

Unstructured Lattices

Lattice: Definition

Definition

Let $r \leq d$ and let $\mathbf{B} \in \mathbb{R}^{d \times r}$ be some matrix with linearly independent columns $\mathbf{b}_1, \ldots, \mathbf{b}_r \in \mathbb{R}^d$. We call a **lattice** an additive group

$$\mathcal{L}(\mathbf{B}) := \{\mathbf{B} \cdot \mathbf{z} : \mathbf{z} \in \mathbb{Z}^r\} = \left\{ \sum_{i \in [r]} z_i \mathbf{b}_i : z_1, \ldots, z_r \in \mathbb{Z} \right\}.$$

We say that the **rank** of this lattice is r.

Lattice: Definition

Definition

Let $r \leq d$ and let $\mathbf{B} \in \mathbb{R}^{d \times r}$ be some matrix with linearly independent columns $\mathbf{b}_1, \ldots, \mathbf{b}_r \in \mathbb{R}^d$. We call a **lattice** an additive group

$$\mathcal{L}(\mathbf{B}) := \{\mathbf{B} \cdot \mathbf{z} : \mathbf{z} \in \mathbb{Z}^r\} = \left\{ \sum_{i \in [r]} z_i \mathbf{b}_i : z_1, \ldots, z_r \in \mathbb{Z} \right\}.$$

We say that the **rank** of this lattice is r.

- We commonly denote lattices by Λ.
- $\mathbb{R}^d$: all $\mathbb{R}$-linear combinations of a basis. Λ: all $\mathbb{Z}$**-linear** combinations of a real basis $\{\mathbf{b}_i\}_{i \in [r]}$.

Lattice: Definition

Definition

Let $r \leq d$ and let $\mathbf{B} \in \mathbb{R}^{d \times r}$ be some matrix with linearly independent columns $\mathbf{b}_1, \ldots, \mathbf{b}_r \in \mathbb{R}^d$. We call a **lattice** an additive group

$$\mathcal{L}(\mathbf{B}) := \{\mathbf{B} \cdot \mathbf{z} : \mathbf{z} \in \mathbb{Z}^r\} = \left\{ \sum_{i \in [r]} z_i \mathbf{b}_i : z_1, \ldots, z_r \in \mathbb{Z} \right\}.$$

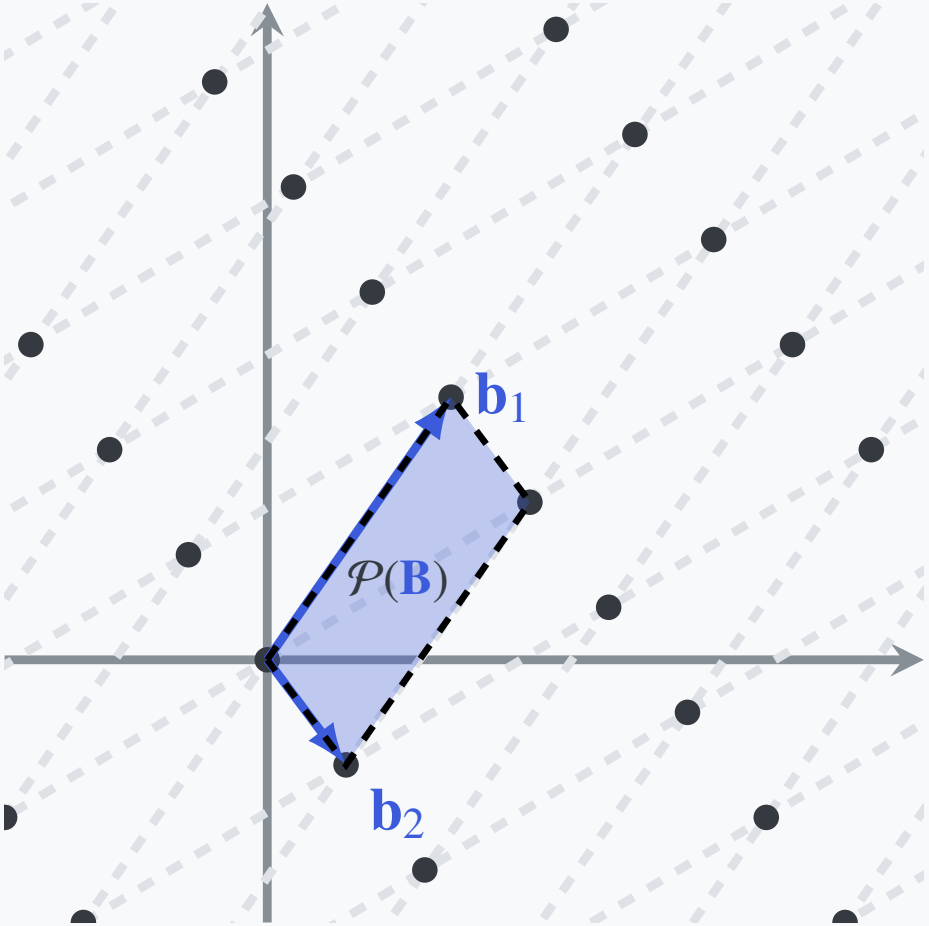
We say that the **rank** of this lattice is r.

- We commonly denote lattices by Λ.
- $\mathbb{R}^d$: all $\mathbb{R}$-linear combinations of a basis. Λ: all $\mathbb{Z}$**-linear** combinations of a real basis $\{\mathbf{b}_i\}_{i \in [r]}$.

Remark

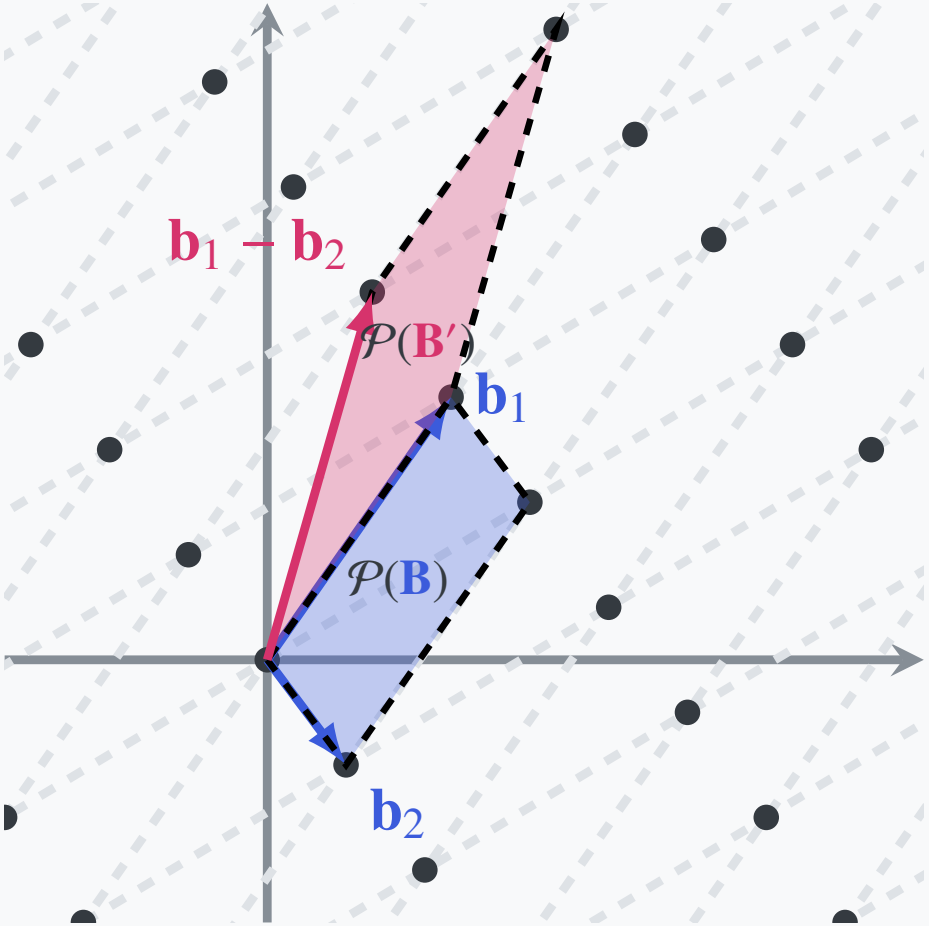
The basis need not lie in $\mathbb{Z}^d$, but in practice it does (rational basis + rescaling). We consider *full-rank* lattices only: $r = d$.

Lattice: Picture



- Lattice Λ of rank 2.
- $\mathbf{B} = \{\mathbf{b}_1, \mathbf{b}_2\}$: a "good" basis, vectors almost orthogonal.

Lattice: Picture



- Lattice Λ of rank 2.
- $\mathbf{B} = \{\mathbf{b}_1, \mathbf{b}_2\}$: a "good" basis, vectors almost orthogonal.
- $\mathbf{B}' = \{\mathbf{b}_1, \mathbf{b}_1 - \mathbf{b}_2\}$: a "worse" basis of the **same** lattice: vectors more parallel, fundamental domain skewed.

Why Lattices?

Lattices appeared in Mathematics long before cryptography (number theory, sphere packings, ...). One of the reasons:

Lemma

Any *discrete* additive subgroup of $\mathbb{R}^d$ is a lattice.

Here, $X \subseteq \mathbb{R}^d$ is *discrete* if $\delta := \inf_{u \neq v} \|u - v\| > 0$ for $u, v \in X$.

Why Lattices?

Lattices appeared in Mathematics long before cryptography (number theory, sphere packings, ...). One of the reasons:

Lemma

Any *discrete* additive subgroup of $\mathbb{R}^d$ is a lattice.

Here, $X \subseteq \mathbb{R}^d$ is *discrete* if $\delta := \inf_{u \neq v} \|u - v\| > 0$ for $u, v \in X$.

Proposition

Let $\mathbb{G}$ be a group generated by $g_1, \ldots, g_d \in \mathbb{G}$. Define the map $\phi : \mathbb{Z}^d \to \mathbb{G}$ as $(z_1, \ldots, z_d) \mapsto z_1 g_1 + \cdots + z_d g_d$. The kernel of this map is a lattice.

Why Lattices?

Lattices appeared in Mathematics long before cryptography (number theory, sphere packings, ...). One of the reasons:

Lemma

Any *discrete* additive subgroup of $\mathbb{R}^d$ is a lattice.

Here, $X \subseteq \mathbb{R}^d$ is *discrete* if $\delta := \inf_{u \neq v} \|u - v\| > 0$ for $u, v \in X$.

Proposition

Let $\mathbb{G}$ be a group generated by $g_1, \ldots, g_d \in \mathbb{G}$. Define the map $\phi : \mathbb{Z}^d \to \mathbb{G}$ as $(z_1, \ldots, z_d) \mapsto z_1 g_1 + \cdots + z_d g_d$. The kernel of this map is a lattice.

Question #1

Verify this proposition.

Fundamental Domain and Covolume

A basis $\mathbf{B}$ of $\Lambda := \mathcal{L}(\mathbf{B})$ is associated with the **fundamental domain**

$$\mathcal{P}(\mathbf{B}) \triangleq \left\{ \sum_{i \in [d]} \theta_i \mathbf{b}_i : \theta_i \in [0, 1) \right\}.$$

- Any $\mathbf{x} \in \mathbb{R}^d$ is uniquely $\mathbf{x} = \mathbf{v} + \mathbf{p}$ with $\mathbf{v} \in \Lambda$, $\mathbf{p} \in \mathcal{P}(\mathbf{B})$: $\mathcal{P}(\mathbf{B})$ represents $\mathbb{R}^d / \Lambda$.
- Long, nearly-parallel vectors $\Rightarrow$ skewed $\mathcal{P}(\mathbf{B}) \Rightarrow$ “bad” basis.

Fundamental Domain and Covolume

A basis $\mathbf{B}$ of $\Lambda := \mathcal{L}(\mathbf{B})$ is associated with the **fundamental domain**

$$\mathcal{P}(\mathbf{B}) \triangleq \left\{ \sum_{i \in [d]} \theta_i \mathbf{b}_i : \theta_i \in [0, 1) \right\}.$$

- Any $\mathbf{x} \in \mathbb{R}^d$ is uniquely $\mathbf{x} = \mathbf{v} + \mathbf{p}$ with $\mathbf{v} \in \Lambda$, $\mathbf{p} \in \mathcal{P}(\mathbf{B})$: $\mathcal{P}(\mathbf{B})$ represents $\mathbb{R}^d / \Lambda$.
- Long, nearly-parallel vectors $\Rightarrow$ skewed $\mathcal{P}(\mathbf{B}) \Rightarrow$ "bad" basis.

Definition (Covolume)

The **covolume** of Λ is $\mathbf{Vol}(\Lambda) \triangleq \mathbf{Vol}(\mathbb{R}^d / \Lambda) = \sqrt{\det \mathbf{B}^\top \mathbf{B}}$.

Fundamental Domain and Covolume

A basis $\mathbf{B}$ of $\Lambda := \mathcal{L}(\mathbf{B})$ is associated with the **fundamental domain**

$$\mathcal{P}(\mathbf{B}) \triangleq \left\{ \sum_{i \in [d]} \theta_i \mathbf{b}_i : \theta_i \in [0, 1) \right\}.$$

- Any $\mathbf{x} \in \mathbb{R}^d$ is uniquely $\mathbf{x} = \mathbf{v} + \mathbf{p}$ with $\mathbf{v} \in \Lambda$, $\mathbf{p} \in \mathcal{P}(\mathbf{B})$: $\mathcal{P}(\mathbf{B})$ represents $\mathbb{R}^d / \Lambda$.
- Long, nearly-parallel vectors $\Rightarrow$ skewed $\mathcal{P}(\mathbf{B}) \Rightarrow$ "bad" basis.

Definition (Covolume)

The **covolume** of Λ is $\mathbf{Vol}(\Lambda) \triangleq \mathbf{Vol}(\mathbb{R}^d / \Lambda) = \sqrt{\det \mathbf{B}^\top \mathbf{B}}$.

Proposition

The covolume $\mathrm{Vol}(\Lambda)$ is an invariant of Λ: it does not depend on the choice of $\mathbf{B}$.

Indeed, two bases satisfy $\mathbf{B}_1 = \mathbf{B}_0 \mathbf{U}$ for a unimodular $\mathbf{U} \in \mathrm{GL}_d(\mathbb{Z})$.

Successive Minima

Denote by $B_\rho := \{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\| < \rho\}$ the open ball of radius ρ.

Definition (Successive minima)

The ith **minimum** $\lambda_i(\Lambda)$ is the radius of the smallest sphere centered at the origin containing i linearly independent lattice vectors:

$$\lambda_i(\Lambda) \triangleq \inf\{\rho \in \mathbb{R}_{\geq 0} : \dim(\text{span}(\Lambda \cap B_\rho)) \geq i\}.$$

Successive Minima

Denote by $B_\rho := \{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\| < \rho\}$ the open ball of radius ρ.

Definition (Successive minima)

The ith **minimum** $\lambda_i(\Lambda)$ is the radius of the smallest sphere centered at the origin containing i linearly independent lattice vectors:

$$\lambda_i(\Lambda) \triangleq \inf\{\rho \in \mathbb{R}_{\geq 0} : \dim(\text{span}(\Lambda \cap B_\rho)) \geq i\}.$$

In particular, the **first minimum** is the length of the shortest vector:

$$\lambda_1(\Lambda) = \min_{\mathbf{x} \in \Lambda \setminus \{\mathbf{0}\}} \|\mathbf{x}\|.$$

Successive Minima

Denote by $B_\rho := \{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\| < \rho\}$ the open ball of radius ρ.

Definition (Successive minima)

The ith **minimum** $\lambda_i(\Lambda)$ is the radius of the smallest sphere centered at the origin containing i linearly independent lattice vectors:

$$\lambda_i(\Lambda) \triangleq \inf\{\rho \in \mathbb{R}_{\geq 0} : \dim(\text{span}(\Lambda \cap B_\rho)) \geq i\}.$$

In particular, the **first minimum** is the length of the shortest vector:

$$\lambda_1(\Lambda) = \min_{\mathbf{x} \in \Lambda \setminus \{\mathbf{0}\}} \|\mathbf{x}\|.$$

Proposition

Let $\Lambda \subseteq \mathbb{R}^d$ be a full-rank lattice. Then, $\lambda_d \geq \ldots \geq \lambda_1 > 0$, and there exist linearly independent $\mathbf{v}_1, \ldots, \mathbf{v}_d \in \Lambda$ with $\|\mathbf{v}_i\| = \lambda_i$.

Shortest Vector Problem (SVP)

Definition (Lattice Generation Algorithm)

LGen(1^n) is a PPT algorithm that outputs a lattice basis $\mathbf{B} \in \mathbb{Z}^{n \times n}$ of rank n.

Shortest Vector Problem (SVP)

Definition (Lattice Generation Algorithm)

$\mathsf{LGen}(1^n)$ is a PPT algorithm that outputs a lattice basis $\mathbf{B} \in \mathbb{Z}^{n \times n}$ of rank n.

Definition (Shortest Vector Problem)

Define $\underline{\text{Game SVP}}^{\mathcal{A}}_{\mathsf{LGen}}$ as follows:

1: Sample the lattice basis $\mathbf{B} \leftarrow \mathsf{LGen}(1^n)$ and send to $\mathcal{A}$;

2: $\mathcal{A}$ outputs some $\mathbf{x} \in \mathbb{Z}^n \setminus \{\mathbf{0}\}$ based on $\mathbf{B}$;

3: **return** 1 iff $\|\mathbf{Bx}\| = \lambda_1(\Lambda)$.

The **SVP assumption** holds if for any PPT adversary $\mathcal{A}$,

$$\mathsf{Adv}^{\mathsf{SVP}}_{\mathcal{A},\mathsf{LGen}}(n) := \Pr\left[\underline{\text{Game SVP}}^{\mathcal{A}}_{\mathsf{LGen}}(n) = 1\right] = \mathsf{negl}(n).$$

Approximate SVP (SVP$_\gamma$)

Finding *the* shortest vector is too strict. Instead, we ask for a small multiple of $\lambda_1(\Lambda)$.

Approximate SVP (SVP_γ)

Finding *the* shortest vector is too strict. Instead, we ask for a small multiple of $\lambda_1(\Lambda)$.

Definition (Approximate Shortest Vector Problem)

Same game, but $\mathcal{A}$ wins iff $\|\mathbf{Bx}\| \leq \gamma\lambda_1(\Lambda)$. The SVP_γ **assumption** holds if $\mathrm{Adv}^{\mathrm{SVP}_\gamma}_{\mathcal{A},\mathsf{LGen}}(n) = \mathsf{negl}(n)$ for any PPT $\mathcal{A}$.

Approximate SVP (SVP_γ)

Finding *the* shortest vector is too strict. Instead, we ask for a small multiple of $\lambda_1(\Lambda)$.

Definition (Approximate Shortest Vector Problem)

Same game, but $\mathcal{A}$ wins iff $\|\mathbf{Bx}\| \leq \gamma\lambda_1(\Lambda)$. The SVP_γ **assumption** holds if $\mathrm{Adv}^{\mathrm{SVP}_\gamma}_{\mathcal{A},\mathsf{LGen}}(n) = \mathrm{negl}(n)$ for any PPT $\mathcal{A}$.

- γ is a function of the rank n: $\gamma = \sqrt{n}$ is "very short", $\gamma = \mathrm{poly}(n)$ is the most interesting case.

Approximate SVP (SVP_γ)

Finding *the* shortest vector is too strict. Instead, we ask for a small multiple of $\lambda_1(\Lambda)$.

Definition (Approximate Shortest Vector Problem)

Same game, but $\mathcal{A}$ wins iff $\|\mathbf{Bx}\| \leq \gamma\lambda_1(\Lambda)$. The SVP_γ **assumption** holds if $\mathrm{Adv}^{\mathrm{SVP}_\gamma}_{\mathcal{A},\mathrm{LGen}}(n) = \mathrm{negl}(n)$ for any PPT $\mathcal{A}$.

- γ is a function of the rank n: $\gamma = \sqrt{n}$ is "very short", $\gamma = \mathrm{poly}(n)$ is the most interesting case.
- For $\gamma = 2^n$ the assumption **fails**: the LLL algorithm solves SVP_γ for $\gamma > 2^{(n \log\log n)/\log n}$.

Approximate SVP (SVP_γ)

Finding *the* shortest vector is too strict. Instead, we ask for a small multiple of $\lambda_1(\Lambda)$.

Definition (Approximate Shortest Vector Problem)

Same game, but $\mathcal{A}$ wins iff $\|\mathbf{Bx}\| \leq \gamma\lambda_1(\Lambda)$. The SVP_γ **assumption** holds if $\mathrm{Adv}^{\mathrm{SVP}_\gamma}_{\mathcal{A},\mathrm{LGen}}(n) = \mathrm{negl}(n)$ for any PPT $\mathcal{A}$.

- γ is a function of the rank n: $\gamma = \sqrt{n}$ is "very short", $\gamma = \mathrm{poly}(n)$ is the most interesting case.
- For $\gamma = 2^n$ the assumption **fails**: the LLL algorithm solves SVP_γ for $\gamma > 2^{(n \log\log n)/\log n}$.
- SIVP_γ: find n linearly independent vectors of length at most $\gamma\lambda_n(\Lambda)$.

Closest Vector and Bounded Distance Decoding

Definition (Approximate Closest Vector Problem, CVP_γ)

Given $\mathbf{B} \leftarrow \mathrm{LGen}(1^n)$ and a target $\mathbf{t} \leftarrow\!\!\$\ \mathbb{Z}^n$, $\mathcal{A}$ outputs $\mathbf{x} \in \mathbb{Z}^n$ and wins iff

$$\|\mathbf{Bx} - \mathbf{t}\| \leq \gamma \|\mathbf{By} - \mathbf{t}\| \quad \text{for all } \mathbf{y} \neq \mathbf{x}.$$

Closest Vector and Bounded Distance Decoding

Definition (Approximate Closest Vector Problem, CVP_γ)

Given $\mathbf{B} \leftarrow \mathrm{LGen}(1^n)$ and a target $\mathbf{t} \leftarrow\!\!\$\ \mathbb{Z}^n$, $\mathcal{A}$ outputs $\mathbf{x} \in \mathbb{Z}^n$ and wins iff

$$\|\mathbf{Bx} - \mathbf{t}\| \leq \gamma \|\mathbf{By} - \mathbf{t}\| \quad \text{for all } \mathbf{y} \neq \mathbf{x}.$$

Sometimes an even looser notion is convenient: the target is "close enough" to Λ (changes in blue).

Definition (Bounded Distance Decoding, BDD_γ)

Given $\mathbf{B}$ and $\mathbf{t}$ with $\mathrm{dist}(\mathbf{t}, \Lambda) < \lambda_1(\Lambda)/2\gamma$, $\mathcal{A}$ outputs $\mathbf{x}$ and wins iff $\mathbf{x} \in \Lambda$ **and** $\|\mathbf{x} - \mathbf{t}\| < \lambda_1(\Lambda)/2\gamma$.

Closest Vector and Bounded Distance Decoding

Definition (Approximate Closest Vector Problem, CVP_γ)

Given $\mathbf{B} \leftarrow \mathrm{LGen}(1^n)$ and a target $\mathbf{t} \hookleftarrow \mathbb{Z}^n$, $\mathcal{A}$ outputs $\mathbf{x} \in \mathbb{Z}^n$ and wins iff

$$\|\mathbf{Bx} - \mathbf{t}\| \leq \gamma \|\mathbf{By} - \mathbf{t}\| \quad \text{for all } \mathbf{y} \neq \mathbf{x}.$$

Sometimes an even looser notion is convenient: the target is "close enough" to Λ (changes in blue).

Definition (Bounded Distance Decoding, BDD_γ)

Given $\mathbf{B}$ and $\mathbf{t}$ with $\mathrm{dist}(\mathbf{t}, \Lambda) < \lambda_1(\Lambda)/2\gamma$, $\mathcal{A}$ outputs $\mathbf{x}$ and wins iff $\mathbf{x} \in \Lambda$ **and** $\|\mathbf{x} - \mathbf{t}\| < \lambda_1(\Lambda)/2\gamma$.

As usual, CVP_γ (BDD_γ) **holds** if every PPT $\mathcal{A}$ has a negligible advantage.

Decisional SVP (GapSVP$_\gamma$)

Most current protocols are based on the *decisional* version of SVP$_\gamma$.

Definition (Decisional Shortest Vector Problem)

Define $\underline{\text{Game GapSVP}}^{\mathcal{A}}_{\gamma,\text{LGen}}$ as follows:

1: Sample a random bit $b \leftarrow\!\$\ \{0,1\}$;
2: If $b = 0$, sample Λ with $\lambda_1(\Lambda) \leq 1$, otherwise with $\lambda_1(\Lambda) > \gamma$;
3: $\mathcal{A}$ outputs some $\widehat{b}$ based on (Λ, γ);
4: **return** 1 iff $b = \widehat{b}$.

GapSVP$_\gamma$ holds if $\left|\Pr[\underline{\text{Game GapSVP}}^{\mathcal{A}}_{\gamma,\text{LGen}}(n) = 1] - \frac{1}{2}\right| = \text{negl}(n)$ for all PPT $\mathcal{A}$.

Decisional SVP (GapSVP$_\gamma$)

Most current protocols are based on the *decisional* version of SVP$_\gamma$.

Definition (Decisional Shortest Vector Problem)

Define $\underline{\text{Game GapSVP}}^{\mathcal{A}}_{\gamma,\text{LGen}}$ as follows:

1: Sample a random bit $b \leftarrow\$ \{0,1\}$;
2: If $b = 0$, sample Λ with $\lambda_1(\Lambda) \leq 1$, otherwise with $\lambda_1(\Lambda) > \gamma$;
3: $\mathcal{A}$ outputs some $\widehat{b}$ based on (Λ, γ);
4: **return** 1 iff $b = \widehat{b}$.

GapSVP$_\gamma$ holds if $\left|\Pr[\underline{\text{Game GapSVP}}^{\mathcal{A}}_{\gamma,\text{LGen}}(n) = 1] - \frac{1}{2}\right| = \text{negl}(n)$ for all PPT $\mathcal{A}$.

Remark

Best attacks on SVP$_\gamma$ (BDGL sieving): $2^{0.292n+o(n)}$ classically and $2^{0.257n+o(n)}$ quantumly. Taking $n \gtrapprox 1024$ typically suffices for ≥ 128 bits of security.

Short Integer Solution (SIS)

SIS: the Idea

Find a *short* non-zero $\mathbf{x} \in \mathbb{Z}^m$ with $\mathbf{A}\mathbf{x} = \mathbf{0}$ over $\mathbb{Z}_q$, for a random $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$.

SIS: the Idea

Find a *short* non-zero $\mathbf{x} \in \mathbb{Z}^m$ with $\mathbf{A}\mathbf{x} = \mathbf{0}$ over $\mathbb{Z}_q$, for a random $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$.

- Without constraints, this is easy: $\mathbf{x} = (q, 0, \ldots, 0)$, or Gaussian elimination.
- We additionally require $\|\mathbf{x}\|_\infty \leq \beta$ for a fixed bound $\beta \ll q$. This makes the problem hard.

SIS: the Idea

Find a *short* non-zero $\mathbf{x} \in \mathbb{Z}^m$ with $\mathbf{Ax} = \mathbf{0}$ over $\mathbb{Z}_q$, for a random $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$.

- Without constraints, this is easy: $\mathbf{x} = (q, 0, \ldots, 0)$, or Gaussian elimination.
- We additionally require $\|\mathbf{x}\|_\infty \leq \beta$ for a fixed bound $\beta \ll q$. This makes the problem hard.

Notation

- q is an (odd) modulus; $x \in \mathbb{Z}_q$ is represented in the range from $-q/2$ to $q/2$.
- $\|x\|_\infty$ is the absolute value of this representative (e.g., $\|q-1\|_\infty = 1$); $\|\mathbf{x}\|_\infty := \max_i \|x_i\|_\infty$ and $\|\mathbf{x}\|_2 := \sqrt{\sum_{i \in [m]} \|x_i\|_\infty^2}$.
- **Short** elements: $S_\eta := \{x \in \mathbb{Z}_q : \|x\|_\infty \leq \eta\}$ with $\eta \ll q$ (think of $\eta \leq 4$ and $q \approx 2^{23}$).

SIS Assumption

Definition (SIS Assumption)

For an adversary $\mathcal{A}$ and parameters (n, m, q, β), define $\underline{\text{Game SIS}}^{\mathcal{A}}_{n,m,q,\beta}$:

1: Sample a matrix $\mathbf{A} \leftarrow\!\$\ \mathbb{Z}_q^{n\times m}$ and send it to $\mathcal{A}$;

2: $\mathcal{A}$ outputs some $\mathbf{x} \in \mathbb{Z}^m$;

3: **return** 1 iff $\mathbf{x} \neq \mathbf{0}$, $\mathbf{Ax} = \mathbf{0} \pmod q$, and $\|\mathbf{x}\|_\infty \leq \beta$.

The $\text{SIS}_{n,m,q,\beta}$ **assumption** holds if for any PPT adversary $\mathcal{A}$,

$$\text{Adv}^{\text{SIS}}_{\mathcal{A}}(n) := \Pr\left[\underline{\text{Game SIS}}^{\mathcal{A}}_{n,m,q,\beta}(n) = 1\right] = \text{negl}(n).$$

SIS Assumption

Definition (SIS Assumption)

For an adversary $\mathcal{A}$ and parameters (n, m, q, β), define $\underline{\text{Game SIS}}^{\mathcal{A}}_{n,m,q,\beta}$:

1: Sample a matrix $\mathbf{A} \leftarrow\$ \mathbb{Z}_q^{n\times m}$ and send it to $\mathcal{A}$;

2: $\mathcal{A}$ outputs some $\mathbf{x} \in \mathbb{Z}^m$;

3: **return** 1 iff $\mathbf{x} \neq \mathbf{0}$, $\mathbf{Ax} = \mathbf{0} \pmod q$, and $\|\mathbf{x}\|_\infty \leq \beta$.

The $\text{SIS}_{n,m,q,\beta}$ **assumption** holds if for any PPT adversary $\mathcal{A}$,

$$\text{Adv}^{\text{SIS}}_{\mathcal{A}}(n) := \Pr\left[\underline{\text{Game SIS}}^{\mathcal{A}}_{n,m,q,\beta}(n) = 1\right] = \text{negl}(n).$$

Remark

Some works use the ℓ^2 norm instead. Since $\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \sqrt{m}\|\mathbf{x}\|_\infty$, the two variants differ by a factor of at most $\sqrt{m}$ in β.

Choosing SIS Parameters

The parameters (n, m, q, β) are not independent:

(i) For $\beta \geq q$ the problem is trivial: $(q, 0, \ldots, 0)$ is a solution.

Choosing SIS Parameters

The parameters (n, m, q, β) are not independent:

(i) For $\beta \geq q$ the problem is trivial: $(q, 0, \ldots, 0)$ is a solution.

(ii) Increasing m can only make SIS **easier**: a solution $\mathbf{x}$ for $\mathbf{A}$ extends to $(\mathbf{x}, \mathbf{0})$ for $[\mathbf{A} \,||\, \mathbf{A}']$. Increasing n can only make SIS **harder**.

Choosing SIS Parameters

The parameters (n, m, q, β) are not independent:

(i) For $\beta \geq q$ the problem is trivial: $(q, 0, \ldots, 0)$ is a solution.

(ii) Increasing m can only make SIS **easier**: a solution $\mathbf{x}$ for $\mathbf{A}$ extends to $(\mathbf{x}, \mathbf{0})$ for $[\mathbf{A} \,||\, \mathbf{A}']$. Increasing n can only make SIS **harder**.

(iii) A short solution exists as soon as $(\beta + 1)^m > q^n$.

Choosing SIS Parameters

The parameters (n, m, q, β) are not independent:

(i) For $\beta \geq q$ the problem is trivial: $(q, 0, \ldots, 0)$ is a solution.

(ii) Increasing m can only make SIS **easier**: a solution $\mathbf{x}$ for $\mathbf{A}$ extends to $(\mathbf{x}, \mathbf{0})$ for $[\mathbf{A} \,||\, \mathbf{A}']$. Increasing n can only make SIS **harder**.

(iii) A short solution exists as soon as $(\beta + 1)^m > q^n$.

Question #2

Why does *(iii)* hold? *Hint:* the map $\mathbf{x} \mapsto \mathbf{Ax}$ sends $(\beta + 1)^m$ vectors $\mathbf{x} \in \{0, \ldots, \beta\}^m$ into a set of size q^n.

Choosing SIS Parameters

The parameters (n, m, q, β) are not independent:

(i) For $\beta \geq q$ the problem is trivial: $(q, 0, \ldots, 0)$ is a solution.

(ii) Increasing m can only make SIS **easier**: a solution $\mathbf{x}$ for $\mathbf{A}$ extends to $(\mathbf{x}, \mathbf{0})$ for $[\mathbf{A} \,||\, \mathbf{A}']$. Increasing n can only make SIS **harder**.

(iii) A short solution exists as soon as $(\beta + 1)^m > q^n$.

Question #2

Why does *(iii)* hold? *Hint:* the map $\mathbf{x} \mapsto \mathbf{A}\mathbf{x}$ sends $(\beta + 1)^m$ vectors $\mathbf{x} \in \{0, \ldots, \beta\}^m$ into a set of size q^n.

By the pigeonhole principle, two distinct $\mathbf{x} \neq \mathbf{x}'$ collide, and $\mathbf{x} - \mathbf{x}'$ is a solution with $\|\mathbf{x} - \mathbf{x}'\|_\infty \leq \beta$. Typically, one takes $m > n \log q / \log(\beta + 1) \gg n$.

SIS as a Lattice Problem

Associate with $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ the set of all integer solutions:

$$\Lambda_q^{\perp}(\mathbf{A}) := \{\mathbf{x} \in \mathbb{Z}^m : \mathbf{A}\mathbf{x} = \mathbf{0} \pmod{q}\}.$$

- Closed under addition and discrete $\Rightarrow$ a lattice.
- $q\mathbb{Z}^m \subseteq \Lambda_q^{\perp}(\mathbf{A}) \subseteq \mathbb{Z}^m$: full rank m, a **q-ary lattice**; $\mathrm{Vol}(\Lambda_q^{\perp}(\mathbf{A})) = q^n$ w.h.p.

SIS as a Lattice Problem

Associate with $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ the set of all integer solutions:

$$\Lambda_q^{\perp}(\mathbf{A}) := \{\mathbf{x} \in \mathbb{Z}^m : \mathbf{A}\mathbf{x} = \mathbf{0} \pmod{q}\}.$$

- Closed under addition and discrete $\Rightarrow$ a lattice.
- $q\mathbb{Z}^m \subseteq \Lambda_q^{\perp}(\mathbf{A}) \subseteq \mathbb{Z}^m$: full rank m, a **q-ary lattice**; $\mathrm{Vol}(\Lambda_q^{\perp}(\mathbf{A})) = q^n$ w.h.p.
- A SIS solution is a short non-zero vector of $\Lambda_q^{\perp}(\mathbf{A})$: SIS is no harder than SVP_γ on this lattice (useful for attacks).

SIS as a Lattice Problem

Associate with $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ the set of all integer solutions:

$$\Lambda_q^{\perp}(\mathbf{A}) := \{\mathbf{x} \in \mathbb{Z}^m : \mathbf{A}\mathbf{x} = \mathbf{0} \pmod{q}\}.$$

- Closed under addition and discrete $\Rightarrow$ a lattice.
- $q\mathbb{Z}^m \subseteq \Lambda_q^{\perp}(\mathbf{A}) \subseteq \mathbb{Z}^m$: full rank m, a **q-ary lattice**; $\mathrm{Vol}(\Lambda_q^{\perp}(\mathbf{A})) = q^n$ w.h.p.
- A SIS solution is a short non-zero vector of $\Lambda_q^{\perp}(\mathbf{A})$: SIS is no harder than SVP_γ on this lattice (useful for attacks).

For security we need the opposite direction. Remarkably (Ajtai), the *average-case* SIS is at least as hard as *worst-case* lattice problems:

Theorem (Informal)

For $m = \mathrm{poly}(n)$, any $\beta > 0$, and any sufficiently large $q \geq \beta \cdot \mathrm{poly}(n)$, solving $\mathrm{SIS}_{n,m,q,\beta}$ is at least as hard as solving GapSVP_γ and SIVP_γ on *arbitrary* n-dimensional lattices for some $\gamma = \beta \cdot \mathrm{poly}(n)$.

Application: Hash Functions

For a random key $\mathbf{A} \leftarrow\!\$\ \mathbb{Z}_q^{n\times m}$ with $2^m > q^n$, define

$$\mathsf{H}_{\mathbf{A}} : \{0,1\}^m \to \mathbb{Z}_q^n, \quad \mathbf{x} \mapsto \mathbf{A}\mathbf{x}.$$

It is compressing: m bits $\mapsto n \log q < m$ bits.

Application: Hash Functions

For a random key $\mathbf{A} \leftarrow\!\$\ \mathbb{Z}_q^{n\times m}$ with $2^m > q^n$, define

$$\mathsf{H}_{\mathbf{A}} : \{0,1\}^m \to \mathbb{Z}_q^n, \quad \mathbf{x} \mapsto \mathbf{A}\mathbf{x}.$$

It is compressing: m bits $\mapsto n \log q < m$ bits.

Proposition

If $\mathsf{SIS}_{n,m,q,1}$ holds, then $\mathsf{H}_{\mathbf{A}}$ is collision-resistant.

Application: Hash Functions

For a random key $\mathbf{A} \leftarrow\!\$\ \mathbb{Z}_q^{n\times m}$ with $2^m > q^n$, define

$$\mathsf{H}_{\mathbf{A}} : \{0,1\}^m \to \mathbb{Z}_q^n, \quad \mathbf{x} \mapsto \mathbf{A}\mathbf{x}.$$

It is compressing: m bits $\mapsto n \log q < m$ bits.

Proposition

If $\mathrm{SIS}_{n,m,q,1}$ holds, then $\mathsf{H}_{\mathbf{A}}$ is collision-resistant.

Proof. Suppose $\mathcal{A}$ finds $\mathbf{x} \neq \mathbf{x}' \in \{0,1\}^m$ with $\mathbf{A}\mathbf{x} = \mathbf{A}\mathbf{x}'$. Then:

- $\mathbf{A}(\mathbf{x} - \mathbf{x}') = \mathbf{0}$ and $\mathbf{x} - \mathbf{x}' \neq \mathbf{0}$;
- each coordinate of $\mathbf{x} - \mathbf{x}'$ lies in S_1.

So $\mathbf{x} - \mathbf{x}'$ solves $\mathrm{SIS}_{n,m,q,1}$ for $\mathbf{A}$. ■

Application: Hash Functions

For a random key $\mathbf{A} \leftarrow\!\$\ \mathbb{Z}_q^{n\times m}$ with $2^m > q^n$, define

$$\mathsf{H}_{\mathbf{A}} : \{0,1\}^m \to \mathbb{Z}_q^n, \quad \mathbf{x} \mapsto \mathbf{A}\mathbf{x}.$$

It is compressing: m bits $\mapsto n \log q < m$ bits.

Proposition

If $\mathrm{SIS}_{n,m,q,1}$ holds, then $\mathsf{H}_{\mathbf{A}}$ is collision-resistant.

Proof. Suppose $\mathcal{A}$ finds $\mathbf{x} \neq \mathbf{x}' \in \{0,1\}^m$ with $\mathbf{A}\mathbf{x} = \mathbf{A}\mathbf{x}'$. Then:

- $\mathbf{A}(\mathbf{x} - \mathbf{x}') = \mathbf{0}$ and $\mathbf{x} - \mathbf{x}' \neq \mathbf{0}$;
- each coordinate of $\mathbf{x} - \mathbf{x}'$ lies in S_1.

So $\mathbf{x} - \mathbf{x}'$ solves $\mathrm{SIS}_{n,m,q,1}$ for $\mathbf{A}$. ■

Remark

Contrary to SHA-256, $\mathsf{H}_{\mathbf{A}}$ is **linear**: $\mathsf{H}_{\mathbf{A}}(\mathbf{x}) + \mathsf{H}_{\mathbf{A}}(\mathbf{x}') = \mathsf{H}_{\mathbf{A}}(\mathbf{x} + \mathbf{x}')$, just like Pedersen commitments.

Ajtai Commitment

Definition (Ajtai Commitment)

The **Ajtai commitment scheme** Π_{Ajtai} over a message space S_δ^n and randomness space S_η^m:

- $\text{Setup}(1^\lambda) \to \text{pp}$: generate $\mathbf{A} \leftarrow\$ \mathbb{Z}_q^{\kappa \times n}$ and $\mathbf{B} \leftarrow\$ \mathbb{Z}_q^{\kappa \times m}$.
- $\text{Com}(\mathbf{x} \in S_\delta^n; \mathbf{r} \in S_\eta^m) \to \mathbf{c} \in \mathbb{Z}_q^\kappa$: output $\mathbf{c} \leftarrow \mathbf{Ax} + \mathbf{Br}$.

Ajtai Commitment

Definition (Ajtai Commitment)

The **Ajtai commitment scheme** Π_{Ajtai} over a message space S_δ^n and randomness space S_η^m:

- Setup$(1^\lambda) \to$ pp: generate $\mathbf{A} \leftarrow\$ \mathbb{Z}_q^{\kappa \times n}$ and $\mathbf{B} \leftarrow\$ \mathbb{Z}_q^{\kappa \times m}$.
- Com$(\mathbf{x} \in S_\delta^n; \mathbf{r} \in S_\eta^m) \to \mathbf{c} \in \mathbb{Z}_q^\kappa$: output $\mathbf{c} \leftarrow \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{r}$.

- **Binding**: two openings $(\mathbf{x}, \mathbf{r}) \neq (\mathbf{x}', \mathbf{r}')$ give the short solution $(\mathbf{x} - \mathbf{x}', \mathbf{r} - \mathbf{r}')$.
- **Hiding** cannot be shown using SIS (we need LWE).
- Additively homomorphic, but **only while messages stay short**: tracking norm growth is the recurring theme of lattice protocols!

Ajtai Commitment

Definition (Ajtai Commitment)

The **Ajtai commitment scheme** Π_{Ajtai} over a message space S_δ^n and randomness space S_η^m:

- Setup$(1^\lambda) \to$ pp: generate $\mathbf{A} \leftarrow\$ \mathbb{Z}_q^{\kappa \times n}$ and $\mathbf{B} \leftarrow\$ \mathbb{Z}_q^{\kappa \times m}$.
- Com$(\mathbf{x} \in S_\delta^n; \mathbf{r} \in S_\eta^m) \to \mathbf{c} \in \mathbb{Z}_q^\kappa$: output $\mathbf{c} \leftarrow \mathbf{Ax} + \mathbf{Br}$.

- **Binding**: two openings $(\mathbf{x}, \mathbf{r}) \neq (\mathbf{x}', \mathbf{r}')$ give the short solution $(\mathbf{x} - \mathbf{x}', \mathbf{r} - \mathbf{r}')$.
- **Hiding** cannot be shown using SIS (we need LWE).
- Additively homomorphic, but **only while messages stay short**: tracking norm growth is the recurring theme of lattice protocols!

Question #3

Which SIS instance does a collision solve? Specify the parameters (n', m', q', β').

Norm Growth and Inhomogeneous SIS

Question #4

Let $\mathbf{x}_1, \ldots, \mathbf{x}_N \in S_1^n$ be committed as $\mathbf{c}_1, \ldots, \mathbf{c}_N$. What SIS bound β is needed so that $\mathbf{c}_1 + \cdots + \mathbf{c}_N$ is still a binding commitment to $\mathbf{x}_1 + \cdots + \mathbf{x}_N$? Assume $\eta \geq 1$.

Norm Growth and Inhomogeneous SIS

Question #4

Let $\mathbf{x}_1, \ldots, \mathbf{x}_N \in S_1^n$ be committed as $\mathbf{c}_1, \ldots, \mathbf{c}_N$. What SIS bound β is needed so that $\mathbf{c}_1 + \cdots + \mathbf{c}_N$ is still a binding commitment to $\mathbf{x}_1 + \cdots + \mathbf{x}_N$? Assume $\eta \geq 1$.

Definition (Inhomogeneous SIS (ISIS))

Given $\mathbf{A} \leftarrow\$ \mathbb{Z}_q^{n \times m}$ and $\mathbf{t} \in \mathbb{Z}_q^n$, find $\mathbf{x}$ with $\|\mathbf{x}\|_\infty \leq \beta$ and $\mathbf{A}\mathbf{x} = \mathbf{t}$.

Norm Growth and Inhomogeneous SIS

Question #4

Let $\mathbf{x}_1, \ldots, \mathbf{x}_N \in S_1^n$ be committed as $\mathbf{c}_1, \ldots, \mathbf{c}_N$. What SIS bound β is needed so that $\mathbf{c}_1 + \cdots + \mathbf{c}_N$ is still a binding commitment to $\mathbf{x}_1 + \cdots + \mathbf{x}_N$? Assume $\eta \geq 1$.

Definition (Inhomogeneous SIS (ISIS))

Given $\mathbf{A} \leftarrow\$ \mathbb{Z}_q^{n \times m}$ and $\mathbf{t} \in \mathbb{Z}_q^n$, find $\mathbf{x}$ with $\|\mathbf{x}\|_\infty \leq \beta$ and $\mathbf{A}\mathbf{x} = \mathbf{t}$.

- ISIS = inverting the one-way function $\mathbf{s} \mapsto \mathbf{A}\mathbf{s}$ for short $\mathbf{s}$: the natural candidate for public-key primitives.
- ISIS is as hard as SIS up to small changes in parameters: a *different* short preimage $\mathbf{x} \neq \mathbf{s}$ yields the SIS solution $\mathbf{x} - \mathbf{s}$.
- We will use it for the Lyubashevsky signature in the next lecture.

Learning With Errors (LWE)

LWE: Motivation

For encryption we need a public key that **looks random** yet hides a secret. Fix a secret $\mathbf{s} \in \mathbb{Z}_q^n$.

LWE: Motivation

For encryption we need a public key that **looks random** yet hides a secret. Fix a secret $\mathbf{s} \in \mathbb{Z}_q^n$.

- Publishing $b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle$ for $\mathbf{a}_i \leftarrow\!\!\$\ \mathbb{Z}_q^n$ is useless: Gaussian elimination recovers $\mathbf{s}$ in $O(n^3)$.

LWE: Motivation

For encryption we need a public key that **looks random** yet hides a secret. Fix a secret $\mathbf{s} \in \mathbb{Z}_q^n$.

- Publishing $b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle$ for $\mathbf{a}_i \leftarrow\!\!\$\ \mathbb{Z}_q^n$ is useless: Gaussian elimination recovers $\mathbf{s}$ in $O(n^3)$.
- LWE (Regev) perturbs each equation by a small error:

$$b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i, \qquad e_i \leftarrow\!\!\$\ \chi,$$

where χ is an *error distribution* over short elements (think of uniform over S_η for $\eta \leq 4$). Gaussian elimination no longer works!

LWE: Motivation

For encryption we need a public key that **looks random** yet hides a secret. Fix a secret $\mathbf{s} \in \mathbb{Z}_q^n$.

- Publishing $b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle$ for $\mathbf{a}_i \leftarrow\!\$\ \mathbb{Z}_q^n$ is useless: Gaussian elimination recovers $\mathbf{s}$ in $O(n^3)$.
- LWE (Regev) perturbs each equation by a small error:

$$b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i, \quad e_i \leftarrow\!\$\ \chi,$$

where χ is an *error distribution* over short elements (think of uniform over S_η for $\eta \leq 4$). Gaussian elimination no longer works!

Definition (LWE Instance)

Stacking m equations, $\mathrm{LWEGen}_{n,m,q,\chi}(\mathbf{s})$ samples $\mathbf{A} \leftarrow\!\$\ \mathbb{Z}_q^{m \times n}$ and $\mathbf{e} \leftarrow\!\$\ \chi^m$, and outputs $(\mathbf{A}, \mathbf{b} := \mathbf{A}\mathbf{s} + \mathbf{e})$.

Search and Decisional LWE

Definition (Search-LWE Assumption)

Sample $\mathbf{s} \hookleftarrow\$ \mathbb{Z}_q^n$, $(\mathbf{A}, \mathbf{b}) \leftarrow \mathsf{LWEGen}_{n,m,q,\chi}(\mathbf{s})$; $\mathcal{A}$ gets $(\mathbf{A}, \mathbf{b})$ and outputs $\widehat{\mathbf{s}}$. The assumption holds if $\mathbf{Adv}_{\mathcal{A}}^{\mathbf{SLWE}}(n) := \Pr[\widehat{\mathbf{s}} = \mathbf{s}] = \mathsf{negl}(n)$ for any PPT $\mathcal{A}$.

Search and Decisional LWE

Definition (Search-LWE Assumption)

Sample $\mathbf{s} \hookleftarrow\$ \mathbb{Z}_q^n$, $(\mathbf{A}, \mathbf{b}) \leftarrow \mathsf{LWEGen}_{n,m,q,\chi}(\mathbf{s})$; $\mathcal{A}$ gets $(\mathbf{A}, \mathbf{b})$ and outputs $\widehat{\mathbf{s}}$. The assumption holds if $\mathbf{Adv}^{\mathbf{SLWE}}_{\mathcal{A}}(n) := \Pr[\widehat{\mathbf{s}} = \mathbf{s}] = \mathsf{negl}(n)$ for any PPT $\mathcal{A}$.

Definition (Decisional-LWE Assumption)

Define $\underline{\text{Game DLWE}}^{\mathcal{A}}_{n,m,q,\chi}$:

1: Sample a random bit $b \hookleftarrow\$ \{0, 1\}$;
2: If $b = 0$, sample $\mathbf{s} \hookleftarrow\$ \mathbb{Z}_q^n$ and $(\mathbf{A}, \mathbf{u}) \leftarrow \mathsf{LWEGen}_{n,m,q,\chi}(\mathbf{s})$; otherwise, $(\mathbf{A}, \mathbf{u}) \hookleftarrow\$ \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m$;
3: $\mathcal{A}$ outputs some $\widehat{b}$ based on $(\mathbf{A}, \mathbf{u})$;
4: **return** 1 iff $\widehat{b} = b$.

DLWE holds if $\mathsf{Adv}^{\mathsf{DLWE}}_{\mathcal{A}}(n) := \left| \Pr\left[\underline{\text{Game DLWE}}^{\mathcal{A}}_{n,m,q,\chi}(n) = 1\right] - \frac{1}{2} \right| = \mathsf{negl}(n)$ for any PPT $\mathcal{A}$.

Search $\Leftrightarrow$ Decision

- Search $\Rightarrow$ decision: find $\mathbf{s}$, check whether $\mathbf{b} - \mathbf{A}\mathbf{s}$ is short.

Search $\Leftrightarrow$ Decision

- Search $\Rightarrow$ decision: find $\mathbf{s}$, check whether $\mathbf{b} - \mathbf{A}\mathbf{s}$ is short.
- Decision $\Rightarrow$ search (Regev, $q = \text{poly}(n)$): recover $\mathbf{s}$ **one coordinate at a time**.

Search $\Leftrightarrow$ Decision

- Search $\Rightarrow$ decision: find $\mathbf{s}$, check whether $\mathbf{b} - \mathbf{A}\mathbf{s}$ is short.
- Decision $\Rightarrow$ search (Regev, $q = \text{poly}(n)$): recover $\mathbf{s}$ **one coordinate at a time**.

Testing whether a guess $g \in \mathbb{Z}_q$ equals s_1:

1: sample $\boldsymbol{\Delta} \leftarrow\!\$\ \mathbb{Z}_q^m$, add it to the first column of $\mathbf{A}$, and add $g\boldsymbol{\Delta}$ to $\mathbf{b}$;

Search $\Leftrightarrow$ Decision

- Search $\Rightarrow$ decision: find $\mathbf{s}$, check whether $\mathbf{b} - \mathbf{A}\mathbf{s}$ is short.
- Decision $\Rightarrow$ search (Regev, $q = \text{poly}(n)$): recover $\mathbf{s}$ **one coordinate at a time**.

Testing whether a guess $g \in \mathbb{Z}_q$ equals s_1:

1: sample $\mathbf{\Delta} \leftarrow\!\$\ \mathbb{Z}_q^m$, add it to the first column of $\mathbf{A}$, and add $g\mathbf{\Delta}$ to $\mathbf{b}$;

2: if $g = s_1$, the modified pair is again an LWE instance with the same secret;

Search $\Leftrightarrow$ Decision

- Search $\Rightarrow$ decision: find $\mathbf{s}$, check whether $\mathbf{b} - \mathbf{A}\mathbf{s}$ is short.
- Decision $\Rightarrow$ search (Regev, $q = \text{poly}(n)$): recover $\mathbf{s}$ **one coordinate at a time**.

Testing whether a guess $g \in \mathbb{Z}_q$ equals s_1:

1: sample $\boldsymbol{\Delta} \leftarrow\!\$\ \mathbb{Z}_q^m$, add it to the first column of $\mathbf{A}$, and add $g\boldsymbol{\Delta}$ to $\mathbf{b}$;

2: if $g = s_1$, the modified pair is again an LWE instance with the same secret;

3: otherwise, $\mathbf{b}$ gets the extra term $(g - s_1)\boldsymbol{\Delta}$, which makes the pair uniform; the distinguisher tells the cases apart (qn calls in total).

Search $\Leftrightarrow$ Decision

- Search $\Rightarrow$ decision: find $\mathbf{s}$, check whether $\mathbf{b} - \mathbf{A}\mathbf{s}$ is short.
- Decision $\Rightarrow$ search (Regev, $q = \text{poly}(n)$): recover $\mathbf{s}$ **one coordinate at a time**.

Testing whether a guess $g \in \mathbb{Z}_q$ equals s_1:

1: sample $\boldsymbol{\Delta} \hookleftarrow \$ \; \mathbb{Z}_q^m$, add it to the first column of $\mathbf{A}$, and add $g\boldsymbol{\Delta}$ to $\mathbf{b}$;
2: if $g = s_1$, the modified pair is again an LWE instance with the same secret;
3: otherwise, $\mathbf{b}$ gets the extra term $(g - s_1)\boldsymbol{\Delta}$, which makes the pair uniform; the distinguisher tells the cases apart (qn calls in total).

Question #5

Check that the modified pair is uniformly distributed when $g \neq s_1$ and q is prime.

Short Secrets and LWE as a Lattice Problem

Short secrets. Practical schemes sample $\mathbf{s} \leftarrow\!\!\$\ \chi^n$. Short-secret LWE with m samples is at least as hard as standard LWE with $m + n$ samples.

Short Secrets and LWE as a Lattice Problem

Short secrets. Practical schemes sample $\mathbf{s} \leftarrow\!\$\ \chi^n$. Short-secret LWE with m samples is at least as hard as standard LWE with $m + n$ samples.

Lattice view. $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$ defines the q-ary lattice

$$\Lambda_q(\mathbf{A}) := \{\mathbf{y} \in \mathbb{Z}^m : \mathbf{y} = \mathbf{A}\mathbf{x} \pmod q \text{ for some } \mathbf{x} \in \mathbb{Z}^n\} = \mathbf{A}\mathbb{Z}^n + q\mathbb{Z}^m.$$

$\mathbf{b}$ is very close to the lattice point $\mathbf{A}\mathbf{s} \bmod q$ (distance $\leq \|\mathbf{e}\|_2 \leq \sqrt{m}\eta$): Search-LWE is an instance of BDD.

Short Secrets and LWE as a Lattice Problem

Short secrets. Practical schemes sample $\mathbf{s} \leftarrow\!\!\$\ \chi^n$. Short-secret LWE with m samples is at least as hard as standard LWE with $m + n$ samples.

Lattice view. $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$ defines the q-ary lattice

$$\Lambda_q(\mathbf{A}) := \{\mathbf{y} \in \mathbb{Z}^m : \mathbf{y} = \mathbf{A}\mathbf{x} \pmod q \text{ for some } \mathbf{x} \in \mathbb{Z}^n\} = \mathbf{A}\mathbb{Z}^n + q\mathbb{Z}^m.$$

$\mathbf{b}$ is very close to the lattice point $\mathbf{A}\mathbf{s} \bmod q$ (distance $\leq \|\mathbf{e}\|_2 \leq \sqrt{m}\eta$): Search-LWE is an instance of BDD.

Theorem

For $m = \text{poly}(n)$, $q \leq 2^{\text{poly}(n)}$, and a (discrete) Gaussian χ of width $\alpha q \geq 2\sqrt{n}$ for $\alpha \in (0, 1)$, solving $\text{DLWE}_{n,m,q,\chi}$ is at least as hard as *quantumly* solving GapSVP_γ and SIVP_γ on arbitrary n-dimensional lattices for some $\gamma = \widetilde{O}(n/\alpha)$.

Application: Regev's Public-Key Encryption

Public key: an LWE instance (looks random by DLWE). Encrypt a bit μ: a random subset-sum of rows of $(\mathbf{A}, \mathbf{b})$, plus $\mu \cdot \lceil q/2 \rfloor$.

Application: Regev's Public-Key Encryption

Public key: an LWE instance (looks random by DLWE). Encrypt a bit μ: a random subset-sum of rows of $(\mathbf{A}, \mathbf{b})$, plus $\mu \cdot \lceil q/2 \rfloor$.

KeyGen(1^λ):

1: $\mathbf{A} \leftarrow\$ \mathbb{Z}_q^{m \times n}$;
2: $\mathbf{s} \leftarrow\$ \mathbb{Z}_q^n$, $\mathbf{e} \leftarrow\$ \chi^m$;
3: $\mathbf{b} \leftarrow \mathbf{A}\mathbf{s} + \mathbf{e}$;
4: pk $\leftarrow (\mathbf{A}, \mathbf{b})$;
5: sk $\leftarrow \mathbf{s}$.

Application: Regev's Public-Key Encryption

Public key: an LWE instance (looks random by DLWE). Encrypt a bit μ: a random subset-sum of rows of $(\mathbf{A}, \mathbf{b})$, plus $\mu \cdot \lceil q/2 \rfloor$.

KeyGen(1^λ):

1: $\mathbf{A} \leftarrow\$ \mathbb{Z}_q^{m \times n}$;
2: $\mathbf{s} \leftarrow\$ \mathbb{Z}_q^n$, $\mathbf{e} \leftarrow\$ \chi^m$;
3: $\mathbf{b} \leftarrow \mathbf{A}\mathbf{s} + \mathbf{e}$;
4: pk $\leftarrow (\mathbf{A}, \mathbf{b})$;
5: sk $\leftarrow \mathbf{s}$.

Enc(pk, $\mu \in \{0,1\}$):

1: $\mathbf{x} \leftarrow\$ \{0,1\}^m$;
2: $\widetilde{\mathbf{a}} \leftarrow \mathbf{A}^\top \mathbf{x}$;
3: $\widetilde{b} \leftarrow \langle \mathbf{b}, \mathbf{x} \rangle + \mu \lceil q/2 \rfloor$;
4: **return** $c = (\widetilde{\mathbf{a}}, \widetilde{b})$.

Application: Regev's Public-Key Encryption

Public key: an LWE instance (looks random by DLWE). Encrypt a bit μ: a random subset-sum of rows of $(\mathbf{A}, \mathbf{b})$, plus $\mu \cdot \lceil q/2 \rfloor$.

KeyGen(1^λ):

1: $\mathbf{A} \leftarrow\$ \mathbb{Z}_q^{m \times n}$;
2: $\mathbf{s} \leftarrow\$ \mathbb{Z}_q^n$, $\mathbf{e} \leftarrow\$ \chi^m$;
3: $\mathbf{b} \leftarrow \mathbf{A}\mathbf{s} + \mathbf{e}$;
4: pk $\leftarrow (\mathbf{A}, \mathbf{b})$;
5: sk $\leftarrow \mathbf{s}$.

Enc(pk, $\mu \in \{0,1\}$):

1: $\mathbf{x} \leftarrow\$ \{0,1\}^m$;
2: $\widetilde{\mathbf{a}} \leftarrow \mathbf{A}^\top \mathbf{x}$;
3: $\widetilde{b} \leftarrow \langle \mathbf{b}, \mathbf{x} \rangle + \mu \lceil q/2 \rfloor$;
4: **return** $c = (\widetilde{\mathbf{a}}, \widetilde{b})$.

Dec(sk, c):

1: Parse $(\widetilde{\mathbf{a}}, \widetilde{b}) \leftarrow c$;
2: $\delta \leftarrow \widetilde{b} - \langle \mathbf{s}, \widetilde{\mathbf{a}} \rangle$;
3: **return** 1 iff $\|\delta\|_\infty \geq q/4$.

Here, $m \geq (n+1) \log q$.

Regev's Scheme: Correctness and Security

Correctness. Expanding δ gives

$$\delta = \mathbf{x}^\top(\mathbf{A}\mathbf{s} + \mathbf{e}) + \mu\lceil q/2 \rfloor - \mathbf{s}^\top\mathbf{A}^\top\mathbf{x} = \langle \mathbf{e}, \mathbf{x} \rangle + \mu\lceil q/2 \rfloor.$$

Since $\mathbf{x}$ is binary, $\|\langle \mathbf{e}, \mathbf{x} \rangle\|_\infty \leq m\eta$. If $m\eta < q/4$, decryption is always correct.

Regev's Scheme: Correctness and Security

Correctness. Expanding δ gives

$$\delta = \mathbf{x}^\top(\mathbf{A}\mathbf{s} + \mathbf{e}) + \mu\lceil q/2 \rfloor - \mathbf{s}^\top\mathbf{A}^\top\mathbf{x} = \langle \mathbf{e}, \mathbf{x} \rangle + \mu\lceil q/2 \rfloor.$$

Since $\mathbf{x}$ is binary, $\|\langle \mathbf{e}, \mathbf{x} \rangle\|_\infty \leq m\eta$. If $m\eta < q/4$, decryption is always correct.

Security (IND-CPA under DLWE), in two steps:

1: by DLWE, replace $\mathbf{b}$ with a uniformly random vector;

2: once $(\mathbf{A}, \mathbf{b})$ is uniform, the leftover hash lemma ($m \geq (n+1)\log q$) makes $(\mathbf{A}^\top\mathbf{x}, \langle \mathbf{b}, \mathbf{x} \rangle)$ statistically close to uniform.

Regev's Scheme: Correctness and Security

Correctness. Expanding δ gives

$$\delta = \mathbf{x}^\top(\mathbf{A}\mathbf{s} + \mathbf{e}) + \mu\lceil q/2 \rfloor - \mathbf{s}^\top\mathbf{A}^\top\mathbf{x} = \langle \mathbf{e}, \mathbf{x} \rangle + \mu\lceil q/2 \rfloor.$$

Since $\mathbf{x}$ is binary, $\|\langle \mathbf{e}, \mathbf{x} \rangle\|_\infty \leq m\eta$. If $m\eta < q/4$, decryption is always correct.

Security (IND-CPA under DLWE), in two steps:

1: by DLWE, replace $\mathbf{b}$ with a uniformly random vector;

2: once $(\mathbf{A}, \mathbf{b})$ is uniform, the leftover hash lemma ($m \geq (n+1)\log q$) makes $(\mathbf{A}^\top\mathbf{x}, \langle \mathbf{b}, \mathbf{x} \rangle)$ statistically close to uniform.

Inefficiency

The public key takes $m(n+1)\log q$ bits, and each *bit* of the message costs $(n+1)\log q$ bits of ciphertext. For $n = 512$, $q \approx 2^{12}$: about 6000 bits per plaintext bit!

Structured Lattices

Why Structure?

- $\mathbf{A}$ has nm elements of $\mathbb{Z}_q$: hundreds of kilobytes, and multiplication by $\mathbf{A}$ costs $O(nm)$.

Why Structure?

- $\mathbf{A}$ has nm elements of $\mathbb{Z}_q$: hundreds of kilobytes, and multiplication by $\mathbf{A}$ costs $O(nm)$.
- Expanding $\mathbf{A}$ from a public seed (e.g., SHAKE) removes the storage cost of $\mathbf{A}$, but $\mathbf{b}$, ciphertexts and computation stay large.

Why Structure?

- $\mathbf{A}$ has nm elements of $\mathbb{Z}_q$: hundreds of kilobytes, and multiplication by $\mathbf{A}$ costs $O(nm)$.
- Expanding $\mathbf{A}$ from a public seed (e.g., SHAKE) removes the storage cost of $\mathbf{A}$, but $\mathbf{b}$, ciphertexts and computation stay large.
- All modern schemes (incl. NIST standards ML-KEM and ML-DSA) replace $\mathbb{Z}_q$ with a polynomial ring $\mathcal{R}_q$: one element is n numbers, yet multiplication by it acts as an $n \times n$ matrix.

Why Structure?

- $\mathbf{A}$ has nm elements of $\mathbb{Z}_q$: hundreds of kilobytes, and multiplication by $\mathbf{A}$ costs $O(nm)$.
- Expanding $\mathbf{A}$ from a public seed (e.g., SHAKE) removes the storage cost of $\mathbf{A}$, but $\mathbf{b}$, ciphertexts and computation stay large.
- All modern schemes (incl. NIST standards ML-KEM and ML-DSA) replace $\mathbb{Z}_q$ with a polynomial ring $\mathcal{R}_q$: one element is n numbers, yet multiplication by it acts as an $n \times n$ matrix.

Definition (Cyclotomic Ring)

Fix n to be a power of two and define

$$\mathcal{R} := \mathbb{Z}[T]/(T^n + 1), \quad \mathcal{R}_q := \mathbb{Z}_q[T]/(T^n + 1).$$

Elements: $f = f_0 + f_1 T + \cdots + f_{n-1} T^{n-1}$. Add coefficient-wise; multiply as polynomials, then reduce with $T^n = -1$. $T^n + 1$ is the $2n$-th *cyclotomic* polynomial.

Multiplication as a Matrix

Coefficient embedding: $\mathrm{vec}(f) := (f_0, \ldots, f_{n-1})^\top \in \mathbb{Z}^n$. Multiplication by a is linear:

$$\mathrm{vec}(a \cdot f) = \mathrm{rot}(a) \cdot \mathrm{vec}(f) \quad \text{for all } f \in \mathcal{R}.$$

Multiplication as a Matrix

Coefficient embedding: $\mathrm{vec}(f) := (f_0, \ldots, f_{n-1})^\top \in \mathbb{Z}^n$. Multiplication by a is linear:

$$\mathrm{vec}(a \cdot f) = \mathrm{rot}(a) \cdot \mathrm{vec}(f) \quad \text{for all } f \in \mathcal{R}.$$

Multiplying by T shifts the coefficients up, the top one wraps around with a minus sign. Hence, $\mathrm{rot}(a)$ is *anti-circulant*:

$$\mathrm{rot}(a) = \begin{bmatrix} a_0 & -a_{n-1} & \ldots & -a_1 \\ a_1 & a_0 & \ldots & -a_2 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n-1} & a_{n-2} & \ldots & a_0 \end{bmatrix}.$$

Multiplication as a Matrix

Coefficient embedding: $\mathrm{vec}(f) := (f_0, \ldots, f_{n-1})^\top \in \mathbb{Z}^n$. Multiplication by a is linear:

$$\mathrm{vec}(a \cdot f) = \mathrm{rot}(a) \cdot \mathrm{vec}(f) \quad \text{for all } f \in \mathcal{R}.$$

Multiplying by T shifts the coefficients up, the top one wraps around with a minus sign. Hence, $\mathrm{rot}(a)$ is *anti-circulant*:

$$\mathrm{rot}(a) = \begin{bmatrix} a_0 & -a_{n-1} & \ldots & -a_1 \\ a_1 & a_0 & \ldots & -a_2 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n-1} & a_{n-2} & \ldots & a_0 \end{bmatrix}.$$

A $k \times \ell$ matrix over $\mathcal{R}_q$ is a structured $kn \times \ell n$ matrix over $\mathbb{Z}_q$ stored in $k\ell n$ elements!

Multiplication as a Matrix

Coefficient embedding: $\mathrm{vec}(f) := (f_0, \ldots, f_{n-1})^\top \in \mathbb{Z}^n$. Multiplication by a is linear:

$$\mathrm{vec}(a \cdot f) = \mathrm{rot}(a) \cdot \mathrm{vec}(f) \quad \text{for all } f \in \mathcal{R}.$$

Multiplying by T shifts the coefficients up, the top one wraps around with a minus sign. Hence, $\mathrm{rot}(a)$ is *anti-circulant*:

$$\mathrm{rot}(a) = \begin{bmatrix} a_0 & -a_{n-1} & \ldots & -a_1 \\ a_1 & a_0 & \ldots & -a_2 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n-1} & a_{n-2} & \ldots & a_0 \end{bmatrix}.$$

A $k \times \ell$ matrix over $\mathcal{R}_q$ is a structured $kn \times \ell n$ matrix over $\mathbb{Z}_q$ stored in $k\ell n$ elements!

Question #6

For $n = 2$, compare $(1 + 2T)(3 - T) \in \mathbb{Z}[T]/(T^2 + 1)$ with $\mathrm{rot}(1 + 2T) \cdot (3, -1)^\top$.

Ideal Lattices

An *ideal* $\mathcal{I} \subseteq \mathcal{R}$: additive subgroup closed under multiplication by $\mathcal{R}$, e.g., $(g) := \{gf : f \in \mathcal{R}\}$.

- $\mathrm{vec}(\mathcal{I}) \subseteq \mathbb{Z}^n$ is a lattice, called an **ideal lattice**.
- $\mathrm{vec}((g)) = \mathcal{L}(\mathrm{rot}(g))$: n coefficients of g describe the entire basis.

Ideal Lattices

An *ideal* $\mathcal{I} \subseteq \mathcal{R}$: additive subgroup closed under multiplication by $\mathcal{R}$, e.g., $(g) := \{gf : f \in \mathcal{R}\}$.

- $\mathrm{vec}(\mathcal{I}) \subseteq \mathbb{Z}^n$ is a lattice, called an **ideal lattice**.
- $\mathrm{vec}((g)) = \mathcal{L}(\mathrm{rot}(g))$: n coefficients of g describe the entire basis.

Example

Let $\mathcal{R} = \mathbb{Z}[T]/(T^2+1)$ and $\mathcal{I} = (1+T)$. Every element of $\mathcal{I}$ has the form $(1+T)(\alpha+\beta T) = (\alpha-\beta)+(\alpha+\beta)T$, so

$$\mathrm{vec}(\mathcal{I}) = \mathcal{L}\left(\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}\right).$$

Ideal Lattices

An *ideal* $\mathcal{I} \subseteq \mathcal{R}$: additive subgroup closed under multiplication by $\mathcal{R}$, e.g., $(g) := \{gf : f \in \mathcal{R}\}$.

- $\mathrm{vec}(\mathcal{I}) \subseteq \mathbb{Z}^n$ is a lattice, called an **ideal lattice**.
- $\mathrm{vec}((g)) = \mathcal{L}(\mathrm{rot}(g))$: n coefficients of g describe the entire basis.

Example

Let $\mathcal{R} = \mathbb{Z}[T]/(T^2 + 1)$ and $\mathcal{I} = (1 + T)$. Every element of $\mathcal{I}$ has the form $(1 + T)(\alpha + \beta T) = (\alpha - \beta) + (\alpha + \beta)T$, so

$$\mathrm{vec}(\mathcal{I}) = \mathcal{L}\left(\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}\right).$$

Strength and weakness

Closed under $\mathrm{vec}(f) \mapsto \mathrm{vec}(Tf)$. SVP is still $2^{\Omega(n)}$, but GapSVP_γ is easy for small polynomial γ, and quantum algorithms solve $\gamma = 2^{\widetilde{O}(\sqrt{n})}$ in poly time.

Module Lattices

A *module* over $\mathcal{R}$: a "vector space" with scalars from $\mathcal{R}$. For submodules of $\mathcal{R}^d$:

$$\mathsf{vec}(f_1, \ldots, f_d) := (\mathsf{vec}(f_1), \ldots, \mathsf{vec}(f_d)) \in \mathbb{Z}^{nd}.$$

The images are **module lattices** of rank d. Norms: $\|\mathbf{f}\|_\infty := \max_i \|f_i\|_\infty$, $\|\mathbf{f}\|_2 := \|\mathsf{vec}(\mathbf{f})\|_2$.

Module Lattices

A *module* over $\mathcal{R}$: a "vector space" with scalars from $\mathcal{R}$. For submodules of $\mathcal{R}^d$:

$$\mathsf{vec}(f_1, \ldots, f_d) := (\mathsf{vec}(f_1), \ldots, \mathsf{vec}(f_d)) \in \mathbb{Z}^{nd}.$$

The images are **module lattices** of rank d. Norms: $\|\mathbf{f}\|_\infty := \max_i \|f_i\|_\infty$, $\|\mathbf{f}\|_2 := \|\mathsf{vec}(\mathbf{f})\|_2$.

Module lattices interpolate between the two worlds:

- $n = 1$: $\mathcal{R} = \mathbb{Z}$, ordinary unstructured lattices;

Module Lattices

A *module* over $\mathcal{R}$: a "vector space" with scalars from $\mathcal{R}$. For submodules of $\mathcal{R}^d$:

$$\mathsf{vec}(f_1, \ldots, f_d) := (\mathsf{vec}(f_1), \ldots, \mathsf{vec}(f_d)) \in \mathbb{Z}^{nd}.$$

The images are **module lattices** of rank d. Norms: $\|\mathbf{f}\|_\infty := \max_i \|f_i\|_\infty$, $\|\mathbf{f}\|_2 := \|\mathsf{vec}(\mathbf{f})\|_2$.

Module lattices interpolate between the two worlds:

- $n = 1$: $\mathcal{R} = \mathbb{Z}$, ordinary unstructured lattices;
- $d = 1$: ideal lattices;

Module Lattices

A *module* over $\mathcal{R}$: a "vector space" with scalars from $\mathcal{R}$. For submodules of $\mathcal{R}^d$:

$$\mathrm{vec}(f_1, \ldots, f_d) := (\mathrm{vec}(f_1), \ldots, \mathrm{vec}(f_d)) \in \mathbb{Z}^{nd}.$$

The images are **module lattices** of rank d. Norms: $\|\mathbf{f}\|_\infty := \max_i \|f_i\|_\infty$, $\|\mathbf{f}\|_2 := \|\mathrm{vec}(\mathbf{f})\|_2$.

Module lattices interpolate between the two worlds:

- $n = 1$: $\mathcal{R} = \mathbb{Z}$, ordinary unstructured lattices;
- $d = 1$: ideal lattices;
- $n > 1$, $d > 1$: lattices of dimension nd, structured only in $n \times n$ blocks.

Module Lattices

A *module* over $\mathcal{R}$: a "vector space" with scalars from $\mathcal{R}$. For submodules of $\mathcal{R}^d$:

$$\mathrm{vec}(f_1, \ldots, f_d) := (\mathrm{vec}(f_1), \ldots, \mathrm{vec}(f_d)) \in \mathbb{Z}^{nd}.$$

The images are **module lattices** of rank d. Norms: $\|\mathbf{f}\|_\infty := \max_i \|f_i\|_\infty$, $\|\mathbf{f}\|_2 := \|\mathrm{vec}(\mathbf{f})\|_2$.

Module lattices interpolate between the two worlds:

- $n = 1$: $\mathcal{R} = \mathbb{Z}$, ordinary unstructured lattices;
- $d = 1$: ideal lattices;
- $n > 1$, $d > 1$: lattices of dimension nd, structured only in $n \times n$ blocks.

Remark

No known attack exploits the module structure for $d \geq 2$.

Module-SIS and Module-LWE

Now χ samples each coefficient independently, and $S_\eta := \{f \in \mathcal{R}_q : \|f\|_\infty \leq \eta\}$.

Definition (Module-SIS Assumption)

$\mathrm{MSIS}_{q,k,\ell,\beta}$: given $\mathbf{A} \leftarrow\!\!\$\, \mathcal{R}_q^{k\times\ell}$, no PPT $\mathcal{A}$ outputs $\mathbf{x} \in \mathcal{R}^\ell$ with $\mathbf{x} \neq \mathbf{0}$, $\mathbf{Ax} = \mathbf{0}$ over $\mathcal{R}_q$, and $\|\mathbf{x}\|_\infty \leq \beta$ (except with negligible probability).

Module-SIS and Module-LWE

Now χ samples each coefficient independently, and $S_\eta := \{f \in \mathcal{R}_q : \|f\|_\infty \leq \eta\}$.

Definition (Module-SIS Assumption)

$\mathrm{MSIS}_{q,k,\ell,\beta}$: given $\mathbf{A} \leftarrow\!\$\, \mathcal{R}_q^{k\times\ell}$, no PPT $\mathcal{A}$ outputs $\mathbf{x} \in \mathcal{R}^\ell$ with $\mathbf{x} \neq \mathbf{0}$, $\mathbf{Ax} = \mathbf{0}$ over $\mathcal{R}_q$, and $\|\mathbf{x}\|_\infty \leq \beta$ (except with negligible probability).

Definition (Module-LWE Assumption)

A **Module-LWE instance** is $(\mathbf{A}, \mathbf{b} := \mathbf{As} + \mathbf{e})$ for $\mathbf{A} \leftarrow\!\$\, \mathcal{R}_q^{k\times\ell}$, $\mathbf{s} \leftarrow\!\$\, \chi^\ell$, $\mathbf{e} \leftarrow\!\$\, \chi^k$.

- $\mathrm{SMLWE}_{q,k,\ell,\chi}$: no PPT adversary recovers $\mathbf{s}$;
- $\mathrm{DMLWE}_{q,k,\ell,\chi}$: no PPT adversary distinguishes $(\mathbf{A}, \mathbf{b})$ from uniform.

Module-SIS and Module-LWE

Now χ samples each coefficient independently, and $S_\eta := \{f \in \mathcal{R}_q : \|f\|_\infty \leq \eta\}$.

Definition (Module-SIS Assumption)

MSIS$_{q,k,\ell,\beta}$: given $\mathbf{A} \leftarrow\!\$\ \mathcal{R}_q^{k\times\ell}$, no PPT $\mathcal{A}$ outputs $\mathbf{x} \in \mathcal{R}^\ell$ with $\mathbf{x} \neq \mathbf{0}$, $\mathbf{Ax} = \mathbf{0}$ over $\mathcal{R}_q$, and $\|\mathbf{x}\|_\infty \leq \beta$ (except with negligible probability).

Definition (Module-LWE Assumption)

A **Module-LWE instance** is $(\mathbf{A}, \mathbf{b} := \mathbf{As} + \mathbf{e})$ for $\mathbf{A} \leftarrow\!\$\ \mathcal{R}_q^{k\times\ell}$, $\mathbf{s} \leftarrow\!\$\ \chi^\ell$, $\mathbf{e} \leftarrow\!\$\ \chi^k$.

- SMLWE$_{q,k,\ell,\chi}$: no PPT adversary recovers $\mathbf{s}$;
- DMLWE$_{q,k,\ell,\chi}$: no PPT adversary distinguishes $(\mathbf{A}, \mathbf{b})$ from uniform.

Ring-SIS: $k = 1$; **Ring-LWE**: $k = \ell = 1$ (a single sample $b = as + e$).
Via rot, Module-SIS is SIS with a structured rot$(\mathbf{A}) \in \mathbb{Z}_q^{kn\times\ell n}$.

Ring-LWE-based Encryption

Encode n message bits as $\mu := \sum_{i \in [n]} \mu_i T^i \in \mathcal{R}_q$ and encrypt all of them at once.

Ring-LWE-based Encryption

Encode n message bits as $\mu := \sum_{i \in [n]} \mu_i T^i \in \mathcal{R}_q$ and encrypt all of them at once.

KeyGen(1^λ):

1: $a \leftarrow\!\!\$\ \mathcal{R}_q$;
2: $s, e \leftarrow\!\!\$\ \chi$;
3: $b \leftarrow as + e$;
4: pk $\leftarrow (a, b)$;
5: sk $\leftarrow s$.

Ring-LWE-based Encryption

Encode n message bits as $\mu := \sum_{i\in[n]} \mu_i T^i \in \mathcal{R}_q$ and encrypt all of them at once.

KeyGen(1^λ):

1: $a \leftarrow\!\$\ \mathcal{R}_q$;
2: $s, e \leftarrow\!\$\ \chi$;
3: $b \leftarrow as + e$;
4: pk $\leftarrow (a, b)$;
5: sk $\leftarrow s$.

Enc(pk, $\boldsymbol{\mu} \in \{0,1\}^n$):

1: $\mu \leftarrow \sum_{i\in[n]} \mu_i T^i$;
2: $r, e_1, e_2 \leftarrow\!\$\ \chi$;
3: $u \leftarrow ar + e_1$;
4: $v \leftarrow br + e_2 + \lceil q/2 \rfloor \mu$;
5: **return** $c = (u, v)$.

Ring-LWE-based Encryption

Encode n message bits as $\mu := \sum_{i\in[n]} \mu_i T^i \in \mathcal{R}_q$ and encrypt all of them at once.

KeyGen(1^λ):

1: $a \leftarrow\$ \mathcal{R}_q$;
2: $s, e \leftarrow\$ \chi$;
3: $b \leftarrow as + e$;
4: pk $\leftarrow (a, b)$;
5: sk $\leftarrow s$.

Enc(pk, $\boldsymbol{\mu} \in \{0,1\}^n$):

1: $\mu \leftarrow \sum_{i\in[n]} \mu_i T^i$;
2: $r, e_1, e_2 \leftarrow\$ \chi$;
3: $u \leftarrow ar + e_1$;
4: $v \leftarrow br + e_2 + \lceil q/2 \rfloor \mu$;
5: **return** $c = (u, v)$.

Dec(sk, c):

1: Parse $(u, v) \leftarrow c$;
2: $\delta \leftarrow v - su$;
3: $\mu_i \leftarrow 1$ iff $\|\delta_i\|_\infty \geq q/4$ for each $i \in [n]$;
4: **return** $(\mu_0, \ldots, \mu_{n-1})$.

Here, χ is a short distribution over $S_\eta \subseteq \mathcal{R}_q$ and δ_i is the i-th coefficient of δ.

Ring-LWE Encryption: Analysis

Correctness. Since $\mathcal{R}_q$ is commutative ($asr = sar$),

$$\delta = (as + e)r + e_2 + \lceil q/2 \rfloor \mu - s(ar + e_1) = er + e_2 - se_1 + \lceil q/2 \rfloor \mu.$$

By $\|fg\|_\infty \leq n\|f\|_\infty \|g\|_\infty$, every coefficient of the error is at most $2n\eta^2 + \eta$. Decryption is correct whenever $2n\eta^2 + \eta < q/4$.

Ring-LWE Encryption: Analysis

Correctness. Since $\mathcal{R}_q$ is commutative ($asr = sar$),

$$\delta = (as + e)r + e_2 + \lceil q/2 \rfloor \mu - s(ar + e_1) = er + e_2 - se_1 + \lceil q/2 \rfloor \mu.$$

By $\|fg\|_\infty \leq n\|f\|_\infty \|g\|_\infty$, every coefficient of the error is at most $2n\eta^2 + \eta$. Decryption is correct whenever $2n\eta^2 + \eta < q/4$.

Security. Decisional Ring-LWE, applied twice: to replace b with uniform, and to replace (u, v) (Ring-LWE samples with secret r).

Ring-LWE Encryption: Analysis

Correctness. Since $\mathcal{R}_q$ is commutative ($asr = sar$),

$$\delta = (as + e)r + e_2 + \lceil q/2 \rfloor \mu - s(ar + e_1) = er + e_2 - se_1 + \lceil q/2 \rfloor \mu.$$

By $\|fg\|_\infty \leq n\|f\|_\infty \|g\|_\infty$, every coefficient of the error is at most $2n\eta^2 + \eta$. Decryption is correct whenever $2n\eta^2 + \eta < q/4$.

Security. Decisional Ring-LWE, applied twice: to replace b with uniform, and to replace (u, v) (Ring-LWE samples with secret r).

Efficiency

Ciphertext: $2n \log q$ bits for n message bits, instead of $n(n+1) \log q$ bits in Regev's scheme. Encryption: a few multiplications in $\mathcal{R}_q$, each $O(n \log n)$ with the NTT.

Questions

Question #7

Compute $(1 + T^3) \cdot T^2$ in the ring $\mathbb{Z}[T]/(T^4 + 1)$.

Questions

Question #7

Compute $(1 + T^3) \cdot T^2$ in the ring $\mathbb{Z}[T]/(T^4 + 1)$.

Question #8

What is the smallest positive integer k such that $T^k = 1$ in the ring $\mathbb{Z}[T]/(T^8 + 1)$?

Questions

Question #7

Compute $(1 + T^3) \cdot T^2$ in the ring $\mathbb{Z}[T]/(T^4 + 1)$.

Question #8

What is the smallest positive integer k such that $T^k = 1$ in the ring $\mathbb{Z}[T]/(T^8 + 1)$?

Question #9

Consider the toy SIS lattice $\Lambda_5^\perp(\mathbf{A}) = \{(x, y) \in \mathbb{Z}^2 : x + 2y \equiv 0 \pmod 5\}$ for $\mathbf{A} = \begin{bmatrix} 1 & 2 \end{bmatrix}$. What is the smallest β for which $\mathrm{SIS}_{1,2,5,\beta}$ has a solution (in the ℓ^∞ norm)?

Questions

Question #7

Compute $(1 + T^3) \cdot T^2$ in the ring $\mathbb{Z}[T]/(T^4 + 1)$.

Question #8

What is the smallest positive integer k such that $T^k = 1$ in the ring $\mathbb{Z}[T]/(T^8 + 1)$?

Question #9

Consider the toy SIS lattice $\Lambda_5^\perp(\mathbf{A}) = \{(x, y) \in \mathbb{Z}^2 : x + 2y \equiv 0 \pmod 5\}$ for $\mathbf{A} = \begin{bmatrix} 1 & 2 \end{bmatrix}$. What is the smallest β for which $\mathrm{SIS}_{1,2,5,\beta}$ has a solution (in the ℓ^∞ norm)?

Question #10

In $\mathcal{R} = \mathbb{Z}[T]/(T^4 + 1)$, define $f^*(T) := f(T^{-1})$. What is the constant coefficient of $f \cdot f^*$ for $f = 1 + 2T + 3T^2$?

Summary

- A lattice is the set of all $\mathbb{Z}$-linear combinations of a basis; many bases describe the same lattice, some “better” than others.

Summary

- A lattice is the set of all $\mathbb{Z}$-linear combinations of a basis; many bases describe the same lattice, some “better” than others.
- Finding short (SVP_γ) or close (CVP_γ, BDD_γ) lattice vectors is believed to be hard, even for quantum computers.

Summary

- A lattice is the set of all $\mathbb{Z}$-linear combinations of a basis; many bases describe the same lattice, some “better” than others.
- Finding short (SVP_γ) or close (CVP_γ, BDD_γ) lattice vectors is believed to be hard, even for quantum computers.
- **SIS**: find a short $\mathbf{x} \neq \mathbf{0}$ with $\mathbf{Ax} = \mathbf{0}$. Gives hash functions and (binding) Ajtai commitments.

Summary

- A lattice is the set of all $\mathbb{Z}$-linear combinations of a basis; many bases describe the same lattice, some "better" than others.
- Finding short (SVP_γ) or close (CVP_γ, BDD_γ) lattice vectors is believed to be hard, even for quantum computers.
- **SIS**: find a short $\mathbf{x} \neq \mathbf{0}$ with $\mathbf{Ax} = \mathbf{0}$. Gives hash functions and (binding) Ajtai commitments.
- **LWE**: $(\mathbf{A}, \mathbf{As} + \mathbf{e})$ looks uniform. Gives public-key encryption.

Summary

- A lattice is the set of all $\mathbb{Z}$-linear combinations of a basis; many bases describe the same lattice, some "better" than others.
- Finding short (SVP_γ) or close (CVP_γ, BDD_γ) lattice vectors is believed to be hard, even for quantum computers.
- **SIS**: find a short $\mathbf{x} \neq \mathbf{0}$ with $\mathbf{Ax} = \mathbf{0}$. Gives hash functions and (binding) Ajtai commitments.
- **LWE**: $(\mathbf{A}, \mathbf{As} + \mathbf{e})$ looks uniform. Gives public-key encryption.
- Both have **worst-case to average-case** reductions.

Summary

- A lattice is the set of all $\mathbb{Z}$-linear combinations of a basis; many bases describe the same lattice, some "better" than others.
- Finding short (SVP_γ) or close (CVP_γ, BDD_γ) lattice vectors is believed to be hard, even for quantum computers.
- **SIS**: find a short $\mathbf{x} \neq \mathbf{0}$ with $\mathbf{A}\mathbf{x} = \mathbf{0}$. Gives hash functions and (binding) Ajtai commitments.
- **LWE**: $(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e})$ looks uniform. Gives public-key encryption.
- Both have **worst-case to average-case** reductions.
- Structured variants over $\mathcal{R}_q = \mathbb{Z}_q[T]/(T^n + 1)$ (Ring/Module-SIS and LWE) make everything efficient.

Thank you for your attention



research.blockstream.com

github.com/BlockstreamResearch/zk-book



Blockstream